

목차 (Table of Contents)

1. 연구 개요
 - 1.1 배경
 - 1.2 연구 목표
 - 1.3 베이스라인 측정 결과
2. 실험 환경
 - 2.1 하드웨어
 - 2.2 소프트웨어
 - 2.3 모델 정보
3. 선행연구 요약
 - 3.1 Alpamayo 관련 연구
 - 3.2 GPU 메모리 최적화 관련 주요 연구
4. 연구 방향 1: On-Demand Layering & 메모리 파이프라이닝
 - 4.1 레이어 구조 분석
 - 4.2 device_map 실험 결과
 - 4.3 PCIe 대역폭 및 전송 성능 측정
 - 4.4 모듈별 전송 실험 결과
 - 4.5 로딩 전략 비교
 - 4.6 전송 방식별 성능 비교 (시나리오 A 상세)
 - 4.7 결론
5. 연구 방향 2: CPU-GPU 스왑 최적화
 - 5.1 핵심 발견
 - 5.2 실험 결과
 - 5.3 FP16 이론적 최적값 (스왑 없는 경우) 산출
 - 5.4 적용 가능성 판단
6. 연구 방향 3: 창의적 접근
 - 6.1 7개 아이디어 분석 요약
 - 6.2 Top 3 권장
 - 6.3 종합 우선순위 매트릭스
7. 종합 분석 및 권장 전략
 - 7.1 3개 연구 방향 비교 종합
 - 7.2 최적 조합 전략 제안 (배제 반영)

7.3 구현 로드맵 (배제 반영)

8. 향후 연구 계획 (배제 반영)

Tier 1: 즉시 실행 가능한 작업

Tier 2: 단기 연구

Tier 3: 중기 연구

부록

A. 전체 실험 파일 목록

B. 생성된 시각화 목록

Alpamayo-R1-10B VRAM 최적화 연구 탐색 보고서

작성일: 2026-02-25 작성자: Claude (자율 연구 탐색)

1. 연구 개요

1.1 배경

NVIDIA Alpamayo-R1-10B는 자율주행을 위한 Vision-Language-Action(VLA) 모델로, 11.08B 파라미터(BF16 기준 22.16 GB)를 가진다. 이 모델의 FP16 추론에는 약 24GB VRAM이 요구되어, 12GB GPU(RTX 3080 Ti) 환경에서는 CUDA Unified Memory의 자동 스와핑에 의존하게 되며, 이로 인해 극심한 성능 저하가 발생한다.

1.2 연구 목표

- Alpamayo-R1-10B를 **12GB VRAM** 환경에서 효율적으로 구동하는 방법론 탐색
- CPU-GPU 메모리 스와핑의 구조적 비효율 원인 규명
- 3개 연구 방향에 대한 실험 기반 적용 가능성 판단
- 최적 조합 전략 도출

1.3 베이스라인 측정 결과

메트릭	FP16 (Baseline)	4-bit 양자화
추론 시간	273.79초	4.91초
Peak VRAM	21.52 GB	8.87 GB
12GB 이내 동작	불가 (Unified Memory)	가능
스와핑 발생	~9.5 GB 초과	없음

FP16 추론 273.79초의 실질적 원인이 메모리 스와핑 오버헤드임을 4-bit 실험으로 확인.

2. 실험 환경

2.1 하드웨어

항목	사양
GPU	NVIDIA GeForce RTX 3080 Ti (12 GB VRAM)
CPU	Intel i7-10700K
System RAM	15.55 GB (15 GB 실효)
Swap	4 GB
PCIe	Gen3 x16 (이론 최대 15.75 GB/s)

2.2 소프트웨어

항목	버전
OS	WSL2 (Kernel 6.6.87.1-microsoft-standard-WSL2)
Python	3.12.12
PyTorch	2.8.0+cu128
CUDA	12.8
Transformers	4.57.1
Accelerate	1.12.0

2.3 모델 정보

항목	내용
모델	nvidia/Alpamayo-R1-10B
총 파라미터	11.08B
FP16/BF16 크기	22.16 GB
아키텍처	Vision Encoder (SigLIP) + VLM (Qwen3-VL-8B) + Expert (Trajectory Decoder)
추론 흐름	Vision Encoding -> VLM CoT 생성 -> Flow Matching (10 Euler steps)

3. 선행연구 요약

3.1 Alpamayo 관련 연구

Alpamayo-R1은 NVIDIA가 개발한 세계 최초의 산업 규모 오픈 추론 VLA 모델이다 (NeurIPS 2025 공개, CES 2026 출시).

항목	내용
핵심 기여	Chain of Causation(CoC) 데이터셋 + 모듈형 VLA + RL 후학습
구성	Cosmos-Reason (8.2B VLM) + Diffusion 궤적 디코더 (2.3B)
성능	궤적 정확도 12% 향상, 근접 조우율 35% 감소, 지연 99ms
산업 적용	Mercedes-Benz CLA, JLR, Lucid Motors, Uber

관련 연구로 ReasonPlan, Drive-R1, AutoVLA 등이 유사한 추론 VLA 접근법을 제안하고 있으며, Cosmos-Reason1(VLM 백본), DeepSeek-R1(GRPO 기반 RL) 등이 기반 기술로 활용된다.

상세: [prior-work-alpamayo.md](#)

3.2 GPU 메모리 최적화 관련 주요 연구

RTSS 2022 Demand Layering 논문을 중심으로 25편의 관련 연구를 조사했다.

분류	핵심 연구	Alpamayo 관련성
Demand Layering	Ji et al. (RTSS 2022)	높음 - 레이어별 순차 로딩으로 96.5% 메모리 절감
모델 오프로딩	FlexGen (ICML 2023), NEO (MLSys 2025)	높음 - LP 기반 최적 오프로딩, KV Cache 오프로딩
파이프라이닝	PIPO (2025), Superpipeline (2024)	매우 높음 - 소비자 GPU에서 검증된 파이프라인
양자화 (X — 배제: 모델 정확도 저하 방식)	AWQ (MLSys 2024), Q-VLM (NeurIPS 2024)	매우 높음 - VLM 특화 후학습 양자화
KV Cache	PagedAttention (SOSP 2023)	높음 - KV Cache 메모리 효율 극대화
VLM 서빙	Nova (2025)	매우 높음 - VLM 다단계 파이프라인 최적화
Diffusers	HuggingFace group_offload + CUDA Stream	매우 높음 - Demand Layering과 유사한 네이티브 기법

상세: [related-research.md](#)

4. 연구 방향 1: On-Demand Layering & 메모리 파이프라이닝

On-Demand Layering(필요한 레이어만 GPU에 올리고 내리기)과 메모리 파이프라이닝(전송과 연산을 동시에 수행하여 전송 오버헤드 은닉)은 동일한 문제의 두 측면이다. 전자는 “무엇을, 어떤 단위로 올릴 것인가”, 후자는 “어떻게 빠르게 올릴 것인가”에 해당한다. 본 절에서 통합하여 분석한다.

4.1 레이어 구조 분석

Alpamayo의 레이어 구조를 정밀 분석한 결과, VLM Language Model이 전체 메모리의 **68.5%**(15.17 GB)를 차지하여 12GB VRAM의 최대 병목임을 확인했다.

컴포넌트	파라미터	메모리 (BF16)	비율
Vision Encoder	576.4M	1.15 GB	5.2%
VLM Language Model	7.58B	15.17 GB	68.5%
VLM LM Head	637.7M	1.28 GB	5.8%
Expert (Trajectory Decoder)	2.28B	4.56 GB	20.6%

4.2 device_map 실험 결과

device_map="auto" 실험: 모델 로드는 성공(8.99 GB VRAM)했으나, **추론 실패**.

`fuse_traj_tokens()` 의 `masked_scatter` 연산에서 디바이스 분산 배치와 비호환 발생.

수동 오프로딩 실험: 동일한 디바이스 불일치 오류로 추론 실패. VLM 단독(16.44 GB)이 12 GB를 초과하여 모듈 단위 순차 로딩도 불가.

→ `device_map` 및 accelerate 기반 자동 오프로딩은 Alpamayo의 커스텀 토큰 처리 (`fuse_traj_tokens`)와 비호환. **커스텀 구현이 필수**.

4.3 PCIe 대역폭 및 전송 성능 측정

측정 항목	수치
PCIe H2D 대역폭 (pageable)	7.77 GB/s
PCIe D2H 대역폭 (pageable)	2.48 GB/s (H2D의 1/3)
VLM 단일 레이어 크기	0.386 GB
레이어당 H2D 전송 시간	0.344s
CUDA 스트림 비동기 프리페치 속도업	1.53x
VLM 레이어별 비동기 파이프라인 속도업 (추정)	4.58x

4.4 모듈별 전송 실험 결과

각 모듈을 개별적으로 CPU→GPU 전송 및 실행한 결과:

모듈	크기 (BF16)	CPU→GPU 전송	GPU 단독 탑재
Vision Encoder	1.15 GB	0.374s	가능
Expert	4.56 GB	0.890s (forward 0.338s)	가능
VLM 전체	17.60 GB	—	불가능 (레이어별 필수)

이론적 순차 로딩 피크 VRAM: ~6.5 GB (Expert + KV Cache가 병목)

4.5 로딩 전략 비교

세 가지 로딩 전략 모두 “사용할 때만 GPU에 올리고, 끝나면 내리는” 동일한 원리이며, 로딩 단위와 모델 변경 여부가 다르다.

시나리오 A: 레이어별 로딩 (BF16, 모델 원본 유지)

Transformer 레이어를 1개씩 GPU에 올리고, 실행 후 내림. 모델은 원본 그대로.

VLM 레이어 0 (386MB) 올림 → 실행 → 내림
 VLM 레이어 1 (386MB) 올림 → 실행 → 내림
 ... (36개 반복)
 Expert 레이어 0 (127MB) 올림 → 실행 → 내림
 ... (36개 반복)

시나리오 B: 모듈별 로딩 (BF16, 모델 원본 유지)

Vision/VLM/Expert 모듈을 통째로 GPU에 올리고, 실행 후 내림. 모델은 원본 그대로.

Vision Encoder (1.15GB) 통째 올림 → 실행 → 내림
 VLM (15.17GB) 통째 올림 → 실행 → 내림 ← 12GB 초과! 불가능
 Expert (4.56GB) 통째 올림 → 실행 → 내림

시나리오 C: INT4 양자화 + 모듈별 로딩 (X — 배제: 양자화 - 모델 정확도 저하)

모델을 INT4로 축소한 뒤, 모듈 통째로 올리고 내림.

Vision INT4 (0.29GB) 올림 → 실행 → 내림
 VLM INT4 (4.11GB) 올림 → 실행 → 내림 ← 12GB 이내 가능
 Expert INT4 (1.14GB) 올림 → 실행 → 내림

시나리오	모델 변경	로딩 단위	피크 VRAM	12GB 실행	실현성
A: 레이어별 로딩 (BF16)	없음	레이어 1개	~1.8 GB	가능	높음 (구현 복잡)
B: 모듈별 로딩 (BF16)	없음	모듈 통째	~18.4 GB	불가능	단순하나 VLM>12GB
C: INT4 + 모듈별 로딩 (X — 배제)	INT4 양자화	모듈 통째	~6.1 GB	가능	매우 높음

4.6 전송 방식별 성능 비교 (시나리오 A 상세)

B는 불가능, C는 배제이므로, 시나리오 A(레이어별 로딩)의 전송 방식에 따른 성능 차이:

전송 방식	설명	피크 VRAM	추정 추론 시간	실현성
동기식	레이어 전송 완료 후 실행	~6.5 GB	~500-800s+	이론적 가능
비동기 (파이프라이닝)	현재 레이어 실행 중 다음 레이어 전송	~6.5 GB	~400-600s+	핵심 연구 방향

동기식:
레이어 i: [=전송=][=실행=]
레이어 i+1: [=전송=][=실행=]

비동기 (파이프라이닝):
레이어 i: [=전송=][=실행=]
레이어 i+1: [=전송=][=실행=]
레이어 i+2: [=전송=][=실행=]
→ 전송 시간이 실행 시간에 의해 은닉됨 (1.53x~4.58x 스피드업)

4.7 결론

- **핵심 경로:** 시나리오 A(레이어별 로딩) + 비동기 파이프라이닝
- Autoregressive 디코딩에서 매 토큰마다 36개 레이어를 순회하므로, **비동기 프리페치로 전송 오버헤드를 은닉하는 것이 관건**
- `device_map` 자동 오프로딩은 Alpamayo 비호환 → 커스텀 구현 필수
- 비동기 프리페치 실측 1.53x, VLM 레이어별 적용 시 최대 4.58x 스피드업 추정

상세: 01-on-demand-layering/analysis.md, 02-memory-pipelining/analysis.md 시각화:
01-on-demand-layering/figures/, 02-memory-pipelining/figures/

5. 연구 방향 2: CPU-GPU 스왑 최적화

5.1 핵심 발견

발견	수치
WSL2 Pageable D2H 감속	4.5x (pinned 대비)
Pinned H2D 대역폭	8.5 GB/s
Pinned D2H 대역폭	11.8 GB/s (H2D보다 빠름!)
최적 전송 청크 크기	2-8 MB (12.4 GB/s, 단일 전송의 1.6x)
12GB 초과 시 접근 시간 급증	26x
WSL2 per-transfer 고정 오버헤드	0.36 ms
cudaMemPrefetchAsync(CPU)	WSL2 미지원

5.2 실험 결과

방향 2 결과 수정: Pageable D2H 2.48 GB/s는 WSL2의 비정상적 감속이 원인. Pinned memory 사용 시 D2H가 H2D보다 1.4배 빠름 (11.8 vs 8.5 GB/s).

273.79초 추론의 근본 원인: 1. 페이지 폴트 기반 4KB~2MB 단위 on-demand 전송 (대역폭 효율 저하) 2. WSL2 Pageable D2H 비정상 (pinned 대비 4.5x 느림) 3. 12GB 초과 접근 시간 26x 급증 4. 양방향 전송 경합 (D2H 2.05x 감속) 5. Transformer attention의 비순차적 접근 패턴

5.3 FP16 이론적 최적값 (스왑 없는 경우) 산출

VRAM이 충분한 환경(24GB+ GPU)에서 스왑 없이 동작할 때의 FP16 추론 시간을 GPU 메모리 대역폭으로부터 산출한다.

전제: Autoregressive 생성(batch_size=1)에서 매 토큰마다 전체 모델 가중치를 VRAM에서 읽어야 하며, 이 과정은 memory-bandwidth-bound이다.

사용 데이터: - RTX 3080 Ti VRAM 대역폭: 912 GB/s - VLM 가중치 (Language Model + LM Head): $15.17 + 1.28 = 16.45$ GB - Expert 가중치: 4.56 GB - Vision Encoder 가중치: 1.15 GB - 생성 토큰 수: 256 - Flow Matching 스텝 수: 10

산출:

단계	가중치 읽기량	반복 횟수	읽기 시간	연산 시간
VLM (토큰 생성)	16.45 GB/토큰	256회	$16.45 / 912 \times 256 \approx 4.62s$	$15.16T / 34.1T \times 256 \approx 0.11s$
Expert (디퓨전)	4.56 GB/스텝	10회	$4.56 / 912 \times 10 \approx 0.05s$	$\sim 0.01s$
Vision Encoder	1.15 GB (1회)	1회	$\sim 0.001s$	compute $\sim 0.15s$
KV Cache 읽기	평균 ~ 0.5 GB/토큰	256회	$\sim 0.15s$	—
합계			$\approx 4.82s$	$\approx 0.27s$
총 이론 추론 시간				$\approx 5.0s$

연산 시간이 무시 가능한 이유 (Memory-bandwidth-bound):

RTX 3080 Ti는 FP16 연산력 34.1 TFLOPS, 메모리 대역폭 912 GB/s이다. VLM 토큰당 비교:
 - 가중치 읽기: $16.45 \text{ GB} / 912 \text{ GB/s} \approx 18.0ms$ (병목) - 연산: $7.58B \times 2 \text{ FLOP} / 34.1 \text{ TFLOPS} \approx 0.44ms$ (읽기의 2.4%)

Batch_size=1의 LLM 추론은 거의 항상 memory-bandwidth-bound이므로, 연산 시간은 가중치 읽기 시간에 완전히 은닉된다.

5.4 적용 가능성 판단

전략	추정 추론 시간	스왑 오버헤드	VRAM	실현성
스왑 없음 (이론적 최적)	~5.0s	0s (0%)	≥21.52 GB 필요	24GB+ GPU 필요
Baseline (Unified Memory)	273.79s	~269s (98%)	21.52 GB (논리)	현재 상태
Pinned Memory 스와핑	~150-200s	~145-195s	~6.5 GB	가능
최적 Granularity (2-8MB)	~130-180s	~125-175s	~6.5 GB	가능
Pinned + 비동기 + 청크 최적화	~80-150s	~75-145s	~6.5 GB	핵심 목표
INT4 + Pinned (X — 배제: 양자화)	~80-120s	~75-115s	~5.5 GB	가장 현실적이나 배제

핵심 수치: 현재 273.79초 중 약 269초(98%)가 순수 스왑 오버헤드. GPU 연산 자체는 ~5초면 충분하며, 스왑 최적화로 이 격차를 얼마나 좁히느냐가 연구의 핵심 성과 지표.

참고: 4-bit 양자화 모델이 스왑 없이 4.91초를 달성한 실측값이 위 이론적 산출(~5.0초)과 일치하여, 산출의 타당성을 뒷받침한다.

결론: Pinned memory 사용이 필수 (특히 D2H). 2-8MB 청크 전송으로 대역폭 1.6x 향상 가능. 네이티브 Linux 환경 전환 시 WSL2 대비 40% 추가 개선 가능.

상세: [03-swap-optimization/analysis.md](#) 시각화: [03-swap-optimization/figures/](#)

6. 연구 방향 3: 창의적 접근

6.1 7개 아이디어 분석 요약

#	아이디어	가능성	효과	구현 난이도	VRAM 절감
1	하이브리드 양자화 (Attn-INT8 + FFN-INT4) (X — 배제: 모델 정확도 저하 방식)	높음	높음	보통	60-75%
2	KV Cache 압축/오프로딩	중간-높음	중간	보통	~270 MB
3	Speculative Decoding	낮음-중간	높음	어려움	중립
4	레이어 가지치기 (X — 배제: 모델 정확도 저하 방식)	중간	중간-높음	보통	17-50%
5	동적 해상도 조절 (X — 배제: 모델 정확도 저하 방식)	높음	중간	쉬움	25-55% (동적)
6	디퓨전 스텝 축소 (10->5)	높음	중간	쉬움	미미
7	Token Merging + Chunked Prefill	중간-높음	중간	보통	~44% (활성화)

6.2 Top 3 권장

순위	아이디어	종합 점수	권장
1	하이브리드 양자화 (Attn-INT8 + FFN-INT4) (X — 배제)	3.00	즉시 실행
2	디퓨전 스텝 축소 (10->5 steps)	2.40	즉시 실행
3	동적 해상도 조절 (min_pixels 축소) (X — 배제)	2.40	즉시 실행

하이브리드 양자화: FFN이 레이어의 80%를 차지하므로 INT4 적용 시 메모리 효율 극대화, Attention은 INT8로 정확도 유지. 실측: VLM 6.03 GB (FP16의 70% 절감).

디퓨전 스텝 축소: `num_inference_steps` 파라미터 변경만으로 적용. Flow Matching 5 스텝 시 디퓨전 단계 2x 가속 (전체 ~1.3x).

동적 해상도: **MIN_PIXELS** 값 수정만으로 적용. 반으로 줄이면 동적 메모리 ~25% 절감, 속도 ~30% 향상.

6.3 종합 우선순위 매트릭스

순위	아이디어	종합 점수	우선순위
1	하이브리드 양자화 (X — 배제)	3.00	즉시 실행
2	디퓨전 스텝 축소	2.40	즉시 실행
3	동적 해상도 조절 (X — 배제)	2.40	즉시 실행
4	KV Cache 압축	1.44	보조적 적용
5	Token Merging + Chunked Prefill	1.44	중기 연구
6	레이어 가지치기 (X — 배제)	0.96	장기 연구
7	Speculative Decoding	0.32	장기 연구

상세: [04-creative-approaches/analysis.md](#) 시각화: [04-creative-approaches/figures/](#)

7. 종합 분석 및 권장 전략

7.1 3개 연구 방향 비교 종합

방향	핵심 결론	피크 VRAM	추론 시간	구현 난이도	실현성
1. On-Demand Layering & 파이프라이닝	레이어별 비동기 로딩이 핵심, 커스텀 구현 필수	~6.5 GB	~400-600s+	매우 높음	핵심 경로
2. 스왑 최적화	Pinned 필수, 2-8MB 최적, D2H 4.5x 개선	~5.5-6.5 GB	~80-200s	높음	가능
3. 창의적 접근	디퓨전 스텝 축소 등 (양자화/해상도: X — 배제)	~6.0-7.5 GB	~3.2-4.0s	보통	부분적

7.2 최적 조합 전략 제안 (배제 반영)

양자화/가지치기/동적 해상도를 배제한 상태에서의 실현 가능한 전략:

권장 조합: 레이어별 비동기 로딩 + Pinned Memory + 2-8MB 청크 전송 + 디퓨전 스텝 축소

조합	Peak VRAM (추정)	추론 시간 (추정)
Baseline (Unified Memory)	21.52 GB	273.79초
레이어별 동기식 로딩	~6.5 GB	~500-800s+
레이어별 비동기 로딩	~6.5 GB	~400-600s+
비동기 로딩 + Pinned + 청크 최적화	~6.5 GB	~130-300s
비동기 로딩 + Pinned + 청크 + 디퓨전 5스텝	~6.5 GB	~100-250s

핵심 메시지: 모델 정확도를 유지하면서 12GB VRAM 내 동작을 위해서는 레이어별 비동기 로딩 + Pinned memory가 필수. Baseline 대비 추론 시간은 스왑 최적화로 단축 가능하나, 4-bit(4.91초) 수준에는 미달.

7.3 구현 로드맵 (배제 반영)

Phase 1: 즉시 실행 (1-2일) 1. 디퓨전 스텝 10→5 변경 및 품질 비교 2. Pinned memory 기반 명시적 스와핑 프로토타입

Phase 2: 단기 최적화 (1주) 3. 2-8MB 청크 전송 최적화 4. 커스텀 레이어별 오프로딩 구현 (generate() 수정) 5. CUDA 스트림 비동기 프리페치 적용

Phase 3: 중기 연구 (2-4주) 6. KV Cache 오프로딩 (압축 없이 CPU 저장/복원) 7. Token Merging 적용 검토 8. Chunked Prefill 구현

8. 향후 연구 계획 (배제 반영)

Tier 1: 즉시 실행 가능한 작업

작업	예상 소요	예상 효과
디퓨전 스텝 축소 (10→5) + 품질 검증	0.5일	디퓨전 단계 2x 가속
Pinned memory 기반 명시적 스와핑 프로토타입	1일	스와핑 시 D2H 4.5x 개선

Tier 2: 단기 연구

작업	예상 소요	예상 효과
2-8MB 청크 전송 최적화	0.5일	대역폭 1.6x 향상
커스텀 레이어별 오프로딩 (generate 수정)	2주	피크 VRAM ~2-3 GB
CUDA 스트림 비동기 프리페치 적용	1주	전송 오버헤드 은닉 (1.53~4.58x)
KV Cache CPU 오프로딩 (압축 없이)	1주	KV Cache VRAM 절감

Tier 3: 중기 연구

작업	예상 소요	예상 효과
Token Merging 적용	1주	시퀀스 30% 축소
Chunked Prefill	1주	활성화 메모리 44% 절감
Speculative Decoding (Layer-skipping)	2-3주	VLM 단계 2-6x 가속
네이티브 Linux 환경 전환	1일	WSL2 오버헤드 40% 제거

부록

A. 전체 실험 파일 목록

연구 방향	디렉토리	주요 파일
선행연구 (Alpamayo)	<code>research/</code>	<code>prior-work-alpamayo.md</code>
관련 연구 (GPU 메모리)	<code>research/</code>	<code>related-research.md</code>
방향 1: On- Demand Layering & 파이 프라이닝	<code>research/01-on- demand-layering/</code> , <code>research/02-memory- pipelining/</code>	<code>analysis.md</code> , <code>analyze_layers.py</code> , <code>test_device_map.py</code> , <code>test_manual_offload.py</code> , <code>test_sequential_offload.py</code> , <code>test_async_transfer.py</code>
방향 2: 스왑 최 적화	<code>research/03-swap- optimization/</code>	<code>analysis.md</code> , <code>analyze_unified_memory.py</code> , <code>test_prefetch.py</code> , <code>test_pinned_transfer.py</code> , <code>analyze_wsl2_overhead.py</code>
방향 3: 창의적 접근	<code>research/04- creative- approaches/</code>	<code>analysis.md</code> , <code>exp01_hybrid_quantization.py</code> ~ <code>exp07_activation_checkpointing.py</code>
통합 보고서	<code>research/</code>	<code>final-report.md</code>

B. 생성된 시각화 목록

방 향	시각화 파일	설명
1	01-on-demand-layering/figures/ 01_memory_distribution.png	서브모듈별 메모리 분포
1	01-on-demand-layering/figures/ 02_device_map_analysis.png	device_map 배치 분석
1	01-on-demand-layering/figures/ 03_strategy_comparison.png	전략 비교
1	01-on-demand-layering/figures/04_per_stage_vram.png	모듈별 VRAM (BF16 vs INT4)
1	01-on-demand-layering/figures/05_pipeline_diagram.png	On-Demand Layering 파이프라인
1	02-memory-pipelining/figures/01_pipeline_diagram.png	추론 파이프라인 구조
1	02-memory-pipelining/figures/02_vram_timeline.png	VRAM 시계열
1	02-memory-pipelining/figures/ 03_strategy_comparison.png	전략 비교
1	02-memory-pipelining/figures/04_pcie_bandwidth.png	PCIe 대역폭 측정
1	02-memory-pipelining/figures/05_layerwise_analysis.png	모듈별 상세 분석
2	03-swap-optimization/figures/ 01_unified_memory_timeline.png	Unified Memory 모니터링
2	03-swap-optimization/figures/ 02_bandwidth_comparison.png	Pinned vs Pageable 대역폭
2	03-swap-optimization/figures/ 03_granularity_bandwidth.png	전송 단위 vs 대역폭
2	03-swap-optimization/figures/04_wsl2_overhead.png	WSL2 오버헤드 분석
2	03-swap-optimization/figures/ 05_strategy_comparison.png	스왑 전략 비교
2	03-swap-optimization/figures/06_transfer_scaling.png	전송 크기별 스케일링
3		하이브리드 양자화 메모리

방 향	시각화 파일	설명
	04-creative-approaches/figures/ 01_hybrid_quantization_memory.png	
3	04-creative-approaches/figures/ 02_kv_cache_analysis.png	KV Cache 분석
3	04-creative-approaches/figures/03_layer_pruning.png	레이어 가지치기
3	04-creative-approaches/figures/ 04_dynamic_resolution.png	동적 해상도
3	04-creative-approaches/figures/05_diffusion_steps.png	디퓨전 스텝 축소
3	04-creative-approaches/figures/06_priority_matrix.png	종합 우선순위 매트릭스
3	04-creative-approaches/figures/ 07_offloading_strategies.png	오프로딩 전략별 Peak VRAM